The Console Slayer

Feladatleírás

A Doom 1993-ban jelent meg, és alapjaiban változtatta meg a videojátékok világát: grafikája és játékmenete az akkori technológiai szinten lenyűgözően részletgazdag volt. A játékban egy űrgárdista, *Doomguy* szerepét veheted át, aki a Mars egyik holdján csapdába esik és démonok seregeivel kell megküzdenie. Több mint 30 év távlatából is inspirálja a megjelenő játékokat, és alapvető hatással volt arra, amit ma az akció- és lövöldözős játékok nyújtanak.

A leírás alapján egy Doom-inspirálta, felülnézetes, kétdimenziós, karakter-alapú játékot készíthetsz el. A játékban *Doomguy*-t kell kijuttatnod a sötét, démonokkal teli folyosókon keresztül a pálya kijáratáig, miközben megküzdesz az utadba kerülő ellenségekkel. A játék elkészítéséhez többségében olyan ismeretekre lesz szükséged, amelyet a Problémamegoldás programozással tárgy keretében már elsajátítottál. Néhány olyan probléma esetén, amely túlmutat a tárgy anyagán (pl. hanghatások lejátszása), valamivel részletesebben mutatjuk be a lehetséges megoldást.

A játék természetesen nagyon sok tekintetben továbbfejleszthető, a kód felépítése pedig sok esetben jelentősen egyszerűsíthető vagy javítható a további félévekben tanult objektumorientált alapelvek (öröklés és polimorfizmus) és koncepciók (interfészek, eseménykezelés) alkalmazásával. Ezekről a leírás utolsó szakaszában teszünk röviden említést.

A dokumentum verziója: 1.1.0

1 Kezdeti lépések

A játékban szereplő objektumok pozíciója

A játék minden objektumának (játékos, démonok, tárgyak) helyét egy kétdimenziós koordinátarendszerben adjuk meg. Hozd létre a `Position` nevű osztályt.

- Az koordináta x és y komponensét tároljuk egy-egy egész típusú mezőben, az értékek legyenek publikusan lekérdezhetők és módosíthatók az `X` és `Y` tulajdonságokon keresztül.
- A mezők kezdőértéke a konstruktoron keresztül adható meg.
- Legyen egy statikus **Add** nevű metódusa, amely két `Position` típusú objektumot, `p_1`-et és `p_2`-t vár paraméterként, és egy új `Position` objektumot ad vissza, amelynek koordinátái a `p_1` és `p_2` objektumok x és y koordinátáinak összegéből adódnak.
- Legyen egy statikus **Distance** metódusa, amely két `Position` típusú objektumot, `p_1`-et és `p_2`-t vár paraméterként, és visszaadja kettőjük (euklideszi) távolságát egy lebegőpontos számként.

Tesztelés

Hozz létre két `Position` példányt, majd állítsd elő az összegüket, illetve számítsd ki a két pozíció távolságát. Ellenőrizd, hogy helyes eredményeket kaptál-e.

A játékban szereplő objektumok megjelenése

A játékban szereplő objektumok megjelenítésekor kirajzolt egyszerű ábrákat (színes karaktereket) *sprite*-nak fogjuk nevezni. Hozd létre a `ConsoleSprite` nevű osztályt.

Egy sprite objektum esetén tároljuk a háttér színét (`Background`), az előtér színét (`Foreground`), illetve a kirajzolandó karaktert (`Glyph`). Mivel a játék a megjelenítéshez a parancssort fogja használni, így a színek leírására használjuk a `ConsoleColor` típust, a kirajzolandó karakter pedig legyen `char`. Mindhárom érték legyen publikusan lekérdezhető, a beállításuk pedig a konstruktor hívásakor történjen meg az átadott argumentumok alapján.

Tesztelés

Hozz létre egy `ConsoleSprite` példányt. Írd ki a képernyőre a példány `Glyph` jellemzőjét, de előtte állítsd be a kiíráshoz használt előtér- és háttérszíneket a `Foreground` és `Background` értékeknek megfelelően.

A játékos

Doomguy jellemzőit egy *állapot-objektumban* fogjuk tárolni. Hozd létre a `Player` osztályt.

- A játékos aktuális pozícióját tároljuk egy `Position` típusú mezőben, amely legyen publikusan lekérdezhető és módosítható egy `Position` nevű tulajdonságon keresztül.
- A játékos kinézetét tároljuk egy `ConsoleSprite` típusú mezőben, amely szintén legyen publikusan lekérdezhető egy `Sprite` nevű tulajdonságon keresztül.
- Az osztály konstruktora egy x és egy y egész értéket vár, amely alapján létrehoz egy új pozíciót, amelyet beállít a játékos aktuális helyzetének. A játékos megjelenítéséhez használt sprite háttere legyen fekete, előtere zöld, az alakzat pedig egy `'O'` karakter.

Tesztelés

Hozz létre egy `Player` példányt. A `ConsoleSprite` tesztjénél leírthoz hasonló módon jelenítsd meg a játékos sprite-ját a `Position`-nek megfelelő helyen a képernyőn.

A játékmotor

A játék megjelenítéséért (renderelés), a játékos és a démonok mozgásáért és interakcióiért (játéklogika és fizikai motor), valamint hanghatásokért, a pályák betöltésért a játékmotor felel. Hozd létre a `Game` nevű osztályt.

- Tároljuk a játékos jellemzőit leíró objektumot egy `Player` típusú mezőben. A mező értéke legyen lekérhető egy `Player` nevű tulajdonságon keresztül.
- Tároljuk a játék aktuális állapotát (fut vagy véget ért) egy logikai típusú mezőben, amely legyen publikusan lekérdezhető és módosítható az `Exited` tulajdonságon keresztül.
- A konstruktor hozza létre a játékos objektumot a $(0, 0)$ pozícióban.
- A **RenderSingleSprite** nevű privát metódus egyetlen sprite-ot rajzol ki a megadott pozícióra. A metódus egy `Position` és egy `ConsoleSprite` típusú paramétert vár. Ellenőrzi, hogy a pozíció az aktuális képernyőn belül van-e (ehhez használjuk a `Console` `WindowWidth` és `WindowHeight` tulajdonságait), és ha igen, akkor erre a pozícióra kiírja a sprite-nak megfelelő színekkel a benne eltárolt karaktert.
- A **RenderGame** nevű privát metódus kirajzolja a játék aktuális állapotát a parancssorra. Előbb a kurzor láthatóságát (`CursorVisible`) hamisra állítja, alapértékre állítja a parancssori színeket (**ResetColor**), törli a képernyőt, majd a **RenderSingleSprite** meghívásával kirajzoltatja a játékos aktuális pozíciójára a játékos

sprite-ját.

- A **UserAction** nevű privát metódus a felhasználói bemenetet (billentyű leütések) fogja kezelni. Elsőként ellenőrzi, van-e lenyomott billentyű (`Console.KeyAvailable`), majd igaz válasz esetén elvégzi a lenyomott billentyűtől függően a szükséges módosításokat (lásd az alábbi kódrészletet).

```
if (Console.KeyAvailable)
{
    ConsoleKeyInfo pressed = Console.ReadKey(true);
    switch (pressed.Key)
    {
        case ConsoleKey.Escape:
            Exited = true;
            break;

        // ...
    }
}
```

  Legyen lehetőség a játékos mozgatására a nyílbillentyűk lenyomásával (ilyenkor egyszerűen módosítsuk a játékos x vagy y koordinátáját), az `Escape` billentyű leütésekor pedig a játék érjen véget (az `Exited` tulajdonság igaz értékre állításával).

- Hozzunk létre egy publikus **Run** nevű metódust, amelynek meghívásával indítható a játék. A metódusban helyezzünk el a `while` ciklust, ami az alábbi lépéseket ismétli mindaddig, amíg a játék `Exited` tulajdonsága hamis értéket ad:
  1. Jelenítse meg a játék aktuális állapotát a **RenderGame** meghívásával.
  2. Kezelje a felhasználói bemenetet a **UserAction** meghívásával.
  3. A `Thread.Sleep(25);` utasítással várakoztassa a végrehajtást 25 ms-ig, így jelenleg nagyjából 40 FPS képkockasebességet kapunk a játékban. A konkrét érték beállításával a parancssor folyamatos törlése és újrarajzolása miatti "villogást" tudjuk valamelyest csökkenteni (opcionális).

Tesztelés

Hozz létre egy `Game` példányt, majd hívd meg a **Run** metódusát. Az előbbi tesztekhez hasonlóan megjelenik a játékos sprite-ja a képernyőn.

2 Játékelemek

A játékelemek típusai és jellemzőik

A játékban megjelenő statikus objektumokat *játékelemeknek* fogjuk nevezni. A játékos és a démonok előre megadott szabályok szerint interakcióba léphetnek a játékelemekkel, amelyek ennek hatására megváltoztathatják a játékos vagy a démon állapotát. Hozd létre a `GameItem` osztályt.

- Tároljuk a játékelem példány pozícióját egy `Position` típusú mezőben, amelynek értéke legyen publikusan lekérdezhető egy `Position` nevű tulajdonságon keresztül.
- Tároljuk a játékelem példányhoz tartozó sprite-ot egy `ConsoleSprite` típusú mezőben, amely szintén legyen publikusan lekérdezhető egy `Sprite` nevű tulajdonságon keresztül.

Megnevezés	Rövid leírás	Kitöltési tényező	Sprite
`Ammo`	Lőszer csomag, ami 5 lövedéket tartalmaz.	0.0	A
`BFGCell`	Lőszer a BFG-hez, ami egy BFG lövedéket tartalmaz.	0.0	B
`Door`	Ajtó, ami lehet zárva vagy nyitva.	1.0 vagy 0.0	/ vagy /
`LevelExit`	Kijárat a pályáról.	1.0	E
`Medikit`	Elsősegély csomag, ami 25 életerőt tartalmaz.	0.0	+
`ToxicWaste`	Mérgező hulladék, csökkenti az életerőt.	0.0	:
`Wall`	Fal, áthatolhatatlan akadály.	1.0	

Table 1: Játékelemek és jellemzőik.

- Definiáljuk a játékelemek típusát egy `ItemType` felsorolással. A játékelemek lehetséges típusait foglalja össze az 1. táblázat. A pontos működésüket a leírás további részében adjuk meg.

```
enum ItemType { Ammo, BFGCell, Door, LevelExit, Medikit, ToxicWaste, Wall }
```

- Tároljuk a játékelem példány típusát egy `ItemType` típusú mezőben, amelynek értéke legyen publikusan lekérdezhető egy `Type` nevű tulajdonságon keresztül.
- Minden játékelem a pozíciójához tartozó térrészt (csempét) bizonyos arányban tölti ki, ezt a kitöltési tényezőt használjuk majd az ütközések vizsgálatakor (például nem léphetünk majd olyan csempére, amelyen egy 1.0 kitöltési tényezőjű játékelem van). Tároljuk a játékelem példány kitöltési tényezőjét egy `double` típusú mezőben, amelynek értéke legyen publikusan lekérdezhető egy `FillingRatio` nevű tulajdonságon keresztül.
- Bizonyos játékelemek (pl. lőszercsomag) a játék során felhasználódhatnak, ami miatt eltűnnek a pályáról. Tároljuk a játékelem példány elérhetőségének állapotát egy logikai típusú mezőben, amelynek értéke legyen publikusan lekérdezhető egy `Available` nevű tulajdonságon keresztül.
- A **SetInitialProperties** nevű paraméter nélküli privát metódus a játékelem típusától függően elvégzi a játékelem sprite-jának és kitöltési tényezőjének beállítását. Használjuk az 1. táblázatban szereplő adatokat.
- Az osztály konstruktora egy x és y értéket, valamint egy játékelem-típust vár paraméterként, ezek alapján beállítja a játékelem pozícióját és típusát. A játékelem kezdetben legyen elérhető (lásd az `Available` tulajdonságot). Hívjuk meg a **SetInitialProperties** metódust a típustól függő kezdeti jellemzők beállításához.
- Az **Interact** nevű publikus metódus a játékelemek állapotváltozását kezeli olyan esetekben, amikor a játékos (vagy egy démon) interakcióba lép az elemmel.
  - Ha a játékelem típusa `Ammo`, `BFGCell` vagy `Medikit`, akkor az interakció hatására a játékelem elérhetősége változzon hamisra (ilyenkor a játékos felszedte az adott játékelemet).
  - Ha a játékelem típusa `Door`, akkor az interakció hatására a játékelem kitöltési tényezője változzon 1.0-ről 0.0-ra (ilyenkor az ajtó kinyílt), vagy 0.0-ról 1.0-re (ilyenkor az ajtó bezárult). A kitöltési tényező módosítása mellett rendeljünk új sprite-ot is a játékelemhez (változtassuk meg az elem színét az 1. táblázatban látható módon).

Játékelemek tárolása, megjelenítése és törlése

Bővítsük a `Game` osztályt az alábbiak szerint.

- Tároljuk az elérhető játékelemeket egy `GameItem` típusú elemekből álló listában. Az üres listát a `Game` konstruktorában hozzuk létre. A lista legyen publikusan lekérdezhető egy `Items` nevű tulajdonságon keresztül.
- Módosítsuk a **RenderGame** metódust úgy, hogy a játékos kirajzolása előtt egy ciklussal bejárja az `Items` listát, és annak minden elemét rajzolja ki a **RenderSingleSprite** metódus hívásával.
- Hozzunk létre egy **CleanUpGameItems** nevű privát metódust, amely törli az `Items` listából azokat az elemeket, amelyek már nem elérhetők (`Available` tulajdonság). Ügyeljünk rá, hogy a lista bejárása közben történő törlés hibát okozhat. Egy lehetséges megoldási stratégia: előbb gyűjtsük le a törlendő elemeket egy külön listába, majd a legyűjtött elemek listáját bejárva töröljük azokat az `Items`-ből.

Végezd el az alábbi módosítást a `Game` osztályon belül.

- A **UserAction** metódusban kezeld le a `D` billentyű lenyomását is. Ennek hatására járd be az `Items` listát, és minden elemére hívd meg az **Interact** metódust.
- A **Run** metódusban hívd meg a **CleanUpGameItems** metódust az **UserAction** után.

Tesztelés

A főprogramban adj az `Items` listához egy `Door`, egy `Medikit`, egy `Ammo` és egy `Wall` játékelemet még a **Run** meghívása előtt. A játék elindítása után a játékossal rá tudsz lépni a játékelemekkel kitöltött csempékre (ilyenkor a játékos sprite-ja eltakarja a játékelemét). A `D` billentyű lenyomásakor az ajtó állapota megváltozik, a felszedhető játékelem pedig eltűnik.

3 Ütközésvizsgálat

A játékos kitöltési tényezője

A `Player` osztályban vegyünk fel egy `double` típusú mezőt, amely a játékos kitöltési tényezőjét adja meg, hasonlóan a játékelemeknél leírtakhoz. A mező értéke legyen publikusan lekérdezhető egy `FillingRatio` nevű tulajdonságon keresztül. A játékos kitöltési tényezőjét állítsuk be a `Player` konstruktorában 0.4-re.

Kitöltési tényezőtől függő mozgás

A játékban egy leegyszerűsített ütközésvizsgálatot fogunk megvalósítani úgy, hogy minden nem statikus játékobjektum (játékos és démonok) mozgása előtt ellenőrizzük, hogy a lépés nem vezet-e ütközéshez.

Bővítsük a `Game` osztályt az alábbiak szerint.

- Hozzunk létre egy **GetGameItemsWithinDistance** nevű privát metódust, amely paraméterként egy `Position` értéket és egy `double` távolságot vár. A metódus gyűjtse le az `Items` listából azokat a játékelemeket, amelyek nincsenek messzebb a paraméterként átadott távolságnál a paraméterként átadott pozícióhoz viszonyítva. A pozíciók távolságának számításához használjuk a `Position` osztály **Distance** statikus metódusát.
- Hozzunk létre egy **GetTotalFillingRatio** nevű privát metódust, amely paraméterként egy `Position` értéket vár. A metódus előbb meghatározza az adott pozícióban lévő játékelemeket a **GetGameItemsWithinDistance** metódus hívásával (az átadott távolság értéke legyen 0), majd kiszámítja és visszaadja a legyűjtött játékelemek kitöltési tényezőinek összegét.

- Hozzunk létre egy **Move** nevű privát metódust, amely paraméterként egy `Player` példányt és egy `Position` értéket vár, az utóbbi a vél pozíció, ide szeretne elmozdulni a játékos. Határozzuk meg az átadott pozícióban lévő játékelemek összesített kitöltési tényezőinek (**GetTotalFillingRatio**) és a játékos kitöltési tényezőjének összegét. Ha ez az összeg nem nagyobb 1.0-nál, akkor állítsuk be a játékosnak az átadott cél pozíciót, egyébként ne történjen semmi. Megjegyzés: a játékos paraméterként felsorolása feleslegesnek tűnik, azonban ezzel egységesebb alakúak lesznek a mozgatást végző metódusok, amikor a démonokat is beépítjük a játékba.

Módosítsuk a `Game` osztályban a **UserAction** metódust úgy, hogy a mozgáshoz rendelt billentyűk lenyomásakor kiszámítjuk a megfelelő cél pozíciót (ehhez használjuk a `Position` osztály **Add** statikus metódusát), majd a játékossal és a kiszámított cél pozícióval hívjuk meg a **Move** metódust.

Tesztelés

Az előbbi tesztet újra lefuttatva már nem tudunk a falat vagy zárt ajtót tartalmazó pozíciókra lépni. Az ajtó nyitása után viszont az oda történő mozgás megengedett.

4 Ellenfelek

Az ellenfelek típusai és jellemzőik

Doomguy ellenfelei a *démonok*, amelyek jellemzőit a játékoshoz hasonlóan állapot-objektumokban tároljuk, minden démonhoz pontosan egy objektum fog tartozni. Hozd létre a `Demon` osztályt.

- A démon aktuális pozícióját tároljuk egy `Position` típusú mezőben, amely legyen publikusan lekérdezhető és módosítható egy `Position` nevű tulajdonságon keresztül.
- A démon kinézetét tároljuk egy `ConsoleSprite` típusú mezőben, amely szintén legyen publikusan lekérdezhető egy `Sprite` nevű tulajdonságon keresztül.
- Definiáljuk a démonok lehetséges fajtáit egy `DemonType` felsorolással. A különféle démonok jellemzőit a 2. táblázat foglalja össze. A viselkedésüket megvalósító logikát a leírás további részében adjuk meg.

```
enum DemonType { Zombieman, Imp, Mancubus }
```

- Tároljuk a démon példány típusát egy `DemonType` típusú mezőben, amelynek értéke legyen publikusan lekérdezhető egy `Type` nevű tulajdonságon keresztül.
- A játékelemekhez hasonlóan tároljuk a démon kitöltési tényezőjét egy `double` típusú mezőben, amelynek értéke legyen publikusan lekérdezhető egy `FillingRatio` nevű tulajdonságon keresztül.
- A démon aktuális életerő-pontját tároljuk egy egész típusú mezőben. Ennek értéke legyen publikusan lekérdezhető és beállítható egy `Health` nevű tulajdonságon keresztül, de beállításkor ne vehessen fel nullánál kisebb értéket.
- A démonok a játék során meghalhatnak, ekkor eltűnnek a pályáról. Tároljuk egy logikai típusú mezőben, hogy a démon életben van-e még. A mező értéke legyen publikusan lekérdezhető egy `Alive` nevű tulajdonságon keresztül.
- A démon képes lesz a játékost észlelni, ha az *látótávon* belül van, illetve képes lesz *támadótávon* belül megtámadni. Tároljuk egy-egy egész típusú mezőben ezeket a távolságokat, amelynek értéke legyen publikusan lekérdezhető rendre a `SightRange` és `AttackRange` nevű tulajdonságokon keresztül.
- A **SetInitialProperties** nevű paraméter nélküli privát metódus a démon típusától függően elvégzi az egység induló jellemzőinek beállítását. Használjuk az 2. táblázatban szereplő adatokat.

Megnevezés	Kitöltési tényező	Életpont	Sebzés	Gyorsaság	Látótáv	Támadótáv	Sprite
`Zombieman`	0.4	20	3 - 15	70	3	1	
`Imp`	0.4	60	3 - 24	93	6	3	
`Mancubus`	0.9	600	8 - 64	70	9	6	

Table 2: Ellenfelek fajtái és jellemzőik.

- Az osztály konstruktora egy x és egy y egész értéket, továbbá egy `DemonType` típusú értéket vár, amelyek alapján beállítja a démon fajtáját és aktuális helyzetét. Kezdetben a démon legyen életben. Hívjuk meg a **SetInitialProperties** metódust a típustól függő kezdeti jellemzők beállításához.

Az ellenfelek tárolása, megjelenítése és törlése

A démonok tárolása, megjelenítése és eltávolítása a játékelemekhez hasonló módon történik. Bővítsük és módosítsuk a `Game` osztályt az alábbiak szerint.

- Tároljuk az ellenfeleket egy `Demon` típusú elemekből álló listában. Az üres listát a `Game` konstruktorában hozzuk létre. A lista legyen publikusan lekérdezhető egy `Demons` nevű tulajdonságon keresztül.
- Módosítsuk a **RenderGame** metódust úgy, hogy a játékelemek kirajzolása után végezze el a démonok kirajzolását is ugyanazon logika szerint, a játékost pedig ezt követően rajzolja ki.
- Hozzunk létre egy **CleanUpDemons** nevű privát metódust, amely törli a `Demons` listából azokat az ellenfeleket, amelyek meghaltak (`Alive` tulajdonság). Használjuk a **CleanUpGameItems**-nél megismert logikát.
- Hozzunk létre egy **GetDemonsWithinDistance** nevű privát metódust, amely paraméterként egy `Position` értéket és egy `double` távolságot vár. A metódus a **GetGameItemsWithinDistance**-hez hasonlóan gyűjtse le a `Demons` listából a megfelelő ellenfeleket.
- A **GetTotalFillingRatio** metódusban vegyük figyelembe az ellenfeleket is, vagyis a metódus az adott pozícióban lévő játékelemek és démonok összesített kitöltési tényezőjét adja vissza.

Tesztelés

A játékelemek teszteléséhez hasonlóan ideiglenesen tedd elérhetővé a `Demons` listát is, majd a főprogramban adj a listához egy-egy démont. A játék elindítása után a játékossal rá tudsz lépni olyan csempékre, amelyek `Zombieman` vagy `Imp` típusú démon van, `Mancubus` típusú démonnal viszont nem tartózkodhat egy csempén a játékos.

Az ellenfelek viselkedését megvalósító logika

Az ellenfelek cselekedeteit irányító szabályokat az ellenfél *logikájának* fogjuk nevezni. Egy démon attól függően végez valamilyen cselekedetet egy adott pillanatban (például mozog vagy támad), hogy a típusára jellemző szabályok mit írnak elő számára. A viselkedést irányító szabályok a démon aktuális állapotát fogják figyelembe venni. Módosítsd a `Demon` osztályt az alábbiak szerint.

- Definiáljuk a démonok lehetséges állapotát egy `DemonStateType` felsorolással.

```
enum DemonStateType { Idle, Move, Attack }
```

- Tároljuk a démon példány állapotát egy `DemonStateType` típusú mezőben, amelynek értéke legyen publikusan lekérdezhető és módosítható egy `State` nevű tulajdonságon keresztül.
- Hozzuk létre az **UpdateState** nevű publikus metódust, amelynek egyetlen paramétere egy `Player` példány. A metódus elvégzi a démon állapotának frissítését a következők szerint. Először határozzuk meg a démon és a játékos távolságát (használjuk most is a `Position` osztály **Distance** statikus metódusát a két példány pozícióinak átadásával). Ha a távolság kisebb, mint a démon támadótávja, a démon váltson `Attack` állapotba, különben ha a távolság kisebb, mint a démon látótávja, váltson `Move` állapotba, különben váltson `Idle` állapotba.
- A konstruktorban állítsuk be a démon kezdőállapotát `Idle` állapotra.

Bővítsd a `Game` osztályt az alábbiak szerint.

- Hozzunk létre a **Move** nevű privát metódus egy olyan túlterhelt változatát, amely paraméterként egy `Demon` példányt és egy `Position` értéket vár. A metódus lényegében megegyezik a játékos elmozdításáért felelős metódussal, a különbség mindössze annyi, hogy ez egy démon példánnyal végzi el ugyanazokat a lépéseket.
- Hozzuk létre a **DemonMoveLogic** nevű privát metódust, amelynek egyetlen paramétere egy démon példány. A metódus a démonra jellemző mozgási szabályt implementálja. Jelöljünk ki valamilyen szabály szerint egy cél pozíciót, majd hívjuk meg a démonok mozgatását végző **Move** metódust ezzel a cél pozícióval. Egy nagyon egyszerű lehetőség például, hogy véletlenszerűen választunk a démon aktuális pozíciójának szomszédai közül (lásd az alábbi kódrészletet), de a későbbiekben ennél kifinomultabb szabályt is implementálunk.

```
int x = demon.Position.X + RNG.Next(-1, 2);    // -1, 0, 1
int y = demon.Position.Y + RNG.Next(-1, 2);
Position targetPosition = new Position(x, y);
```

- Hozzuk létre a **UpdateDemons** nevű privát metódust, amely a `Demons` listát bejárva először frissíti a démon állapotát az **UpdateState** meghívásával, majd lekérdezi a példány `State` jellemzőjét, és elvégzi az állapotnak megfelelő cselekvést. Ebben a pillanatban ez azt jelenti, hogy ha a démon `Move` állapotban van, akkor hívjuk meg a **DemonMoveLogic** metódust az aktuális példányt átadva paraméterként. Hamarosan megvalósítjuk a támadást is, az ehhez tartozó cselekvést majd szintén ebben a metódusban fogjuk hívni.
- Módosítsuk a **Run** metódust úgy, hogy a kirajzolás előtt előbb kezelje az ellenfelek állapotbeállítását az **UpdateDemons** meghívásával, valamint törölje a halott démonokat a **CleanUpDemons** hívásával.

Tesztelés

Az előbbi tesztet újra lefuttatva a démonok mozogni kezdenek, amint megfelelő távolságba kerül hozzájuk a játékos. Ha támadótávon belül vagyunk, akkor jelenleg látszólag semmi sem történik, de ekkor `Attack` állapotba lépnek. A játék persze ebben a formában élvezhetetlen, mivel a démonok túl gyorsan mozognak.

5 Harcrendszer

Harc során a játékos vagy démon a támadótávján belül lévő ellenfeleknek a távolságtól függően valamekkora sebzést okoz. A harcrendszer megvalósítása során az egyszerűségre helyezzük a hangsúlyt, ezért például nem veszünk figyelembe irányt a lövéskor (vagyis minden, a támadótávon belül lévő ellenfél sebződhet), illetve a megközelítés bizonyos esetekben teljesen valószerűtlen működést eredményez (például az ellenfél falon keresztül is sebződik). Ezek korrekciójára javaslatot teszünk a leírás végén, de a megvalósítás módját nem részletezzük.

A játékos állapotjellemzőinek bővítése

Doomguy alapértelmezett fegyvere a *super shotgun*, amivel a rendelkezésére álló lőszer mennyiségétől függően tud csak lőni. Harc közben megsebezheti őt az ellenfél, viszont minden elpusztított démonért *tapasztalati pontokat* kap, amelyek befolyásolják a maximális életerejét és a lőszer kapacitását. Bővítsd a `Player` osztályt az alábbiak szerint.

- Tároljuk a játékos aktuális életerejét és lőszer mennyiségét egy-egy egész típusú mezőben, amelyek értéke legyen publikusan lekérdezhető rendre a `Health` és `Ammo` nevű tulajdonságokon keresztül.
- Tároljuk egy logikai mezőben, hogy a játékos életben van-e még, a mező értéke legyen publikusan lekérdezhető egy `Alive` nevű tulajdonságon keresztül.
- Tároljuk a játékos tapasztalati pontjait egy egész típusú mezőben, amelynek értéke legyen publikusan lekérdezhető egy `CombatPoints` nevű tulajdonságon keresztül.
- Hozzunk létre egy `MaxHealth` és egy `MaxAmmo` nevű publikus tulajdonságot, amelyek rendre a játékos életerejének és lőszer kapacitásának elméleti maximumait adják meg a

$$\text{MaxHealth} = 100 + \frac{\text{CombatPoints}}{10} \qquad \text{és} \qquad \text{MaxAmmo} = 10 + \frac{\text{CombatPoints}}{50}$$

  képletek alapján.
- Az életerőt legyen lehetőség módosítani is a `Health` nevű tulajdonságon keresztül, de nem nőhet `MaxHealth` értékénél nagyobbra, és nem vehet fel negatív értéket sem. Ugyanígy legyen lehetőség a lőszer mennyiségét is beállítani az `Ammo` tulajdonságon keresztül, de ennek értéke sem nőhet `MaxAmmo` értékénél nagyobbra, és nem vehet fel negatív értéket sem.
- Tároljuk a játékos látótávolságát egy egész típusú mezőben, amelyek értéke legyen publikusan lekérdezhető a `SightRange` nevű tulajdonságon keresztül.
- Hozzunk létre egy **Shoot** nevű publikus metódust, amely eggyel csökkenti a játékos aktuális lőszer mennyiségét.
- Hozzunk létre egy **AddCombatPoints** nevű publikus metódust, amely a paraméterként átadott értékkel növeli a játékos tapasztalati pontjait.
- Hozzunk létre egy **TakeDamage** nevű publikus metódust, amely a paraméterként átadott egész értékkel csökkenti a játékos aktuális életerejét. Ha az életerő nullára csökken, az `Alive` tulajdonsághoz kapcsolódó adattag értékét `false`-ra állítja.
- A konstruktorban állítsuk 100-ra a kezdő életerőt, 8-ra a kezdő látótávot, `true`-ra a játékos `Alive` tulajdonságához kapcsolódó adattagot, illetve beállíthatunk egy kezdő lőszer mennyiséget is.

Bővítsd a `Demon` osztályt az alábbiak szerint.

- Hozzunk létre egy **TakeDamage** nevű publikus metódust, amely a paraméterként átadott egész értékkel csökkenti a démon aktuális életerejét. Ha az életerő nullára csökken, az `Alive` tulajdonsághoz kapcsolódó adattag értékét hamisra állítja.

A játékos támadásának megvalósítása

Módosítsd a `Game` osztályt az alábbiak szerint.

- Hozzunk létre egy **PlayerAttackLogic** nevű publikus metódust. Első lépésben ellenőrizzük az `Ammo` tulajdonság lekérdezésével, hogy a játékos rendelkezik-e valamennyi lőszerrel. Ha igen, akkor hívjuk meg a játékos **Shoot** metódusát, majd gyűjtsük le a látótávolságán belül lévő démonokat a **GetDemonsWithinDistance**

metódussal, és járjuk be az így kapott listát. Minden démonra számítsuk ki a játékostól vett távolságát, majd határozzuk meg a lövéssel okozott sebződést a

$$\text{damage} = \frac{2 \times U(35, 105)}{1 + \text{distance}}$$

képlettel, ahol $U(35, 105)$ egy 35 és 105 közötti, véletlenszerűen generált érték (használjuk a `Random`.**Next** metódust). A hányadost képezhetjük egészek közötti osztással. Hívjuk meg a démon **TakeDamage** metódusát a kapott sebződést átadva. Ha a támadás után a démon már halott (ez lekérdezhető az `Alive` tulajdonságból), akkor növeljük meg a játékos tapasztalati pontjait az **AddCombatPoints** meghívásával (például `Zombieman` esetén 1-gyel, `Imp` esetén 3-mal, `Mancubus` esetén 10-zel).

- A **UserAction** metódust bővíts úgy, hogy az kezelje az `A` billentyű lenyomását is. Ennek hatására hívd meg a **PlayerAttackLogic** metódust.

Tesztelés

Az előbbi tesztet újra lefuttatva a démonok közelében leadott lövések hatására előbb-utóbb (a véletlenszerűen generált sebzésektől függően) a démonok eltűnnek a képernyőről.

Az ellenfelek támadásának megvalósítása

Az ellenfelek támadását a játékos támadásához nagyon hasonló módon valósítjuk meg. Módosítsd a `Game` osztályt az alábbiak szerint.

- Hozzunk létre egy **DemonAttackLogic** nevű privát metódust, amelynek egyetlen paramétere egy démon. Számítsuk ki a démon játékostól vett távolságát, majd határozzuk meg a támadással okozott sebződést az előbbi képletet használva, de a véletlen érték generálásakor használjuk a 2. táblázatban megadott sebzés értékeket az alsó és felső határok beállításához. Hívjuk meg a játékos **TakeDamage** metódusát a kapott sebződést átadva.
- Az **UpdateDemons** metódusban kezeljük a démonok `Attack` állapotát is, ebben az esetben hívjuk meg a **DemonAttackLogic** metódust.
- A **Run** metódusban a ciklusfeltételt módosítsuk úgy, hogy akkor is kilépjünk, ha a játékos `Alive` tulajdonsága `false` értéket kap, ezzel leállítva a játékot.

Tesztelés

Ha az előbbi tesztet újra lefuttatjuk, a démonok közelébe érve a játék szinte azonnal véget ér, mivel a játékos életpontjai a villámgyors egymás utáni sebződés miatt nullára csökken.

6 Játékmotor refaktorálása

A játékmotort megvalósító `Game` osztályunkban lévő kódsorok száma mostanra valószínűleg több száz, ami nehezítheti az áttekinthetőséget. Jelenleg ez az osztály felelős a játékelemek és ellenfelek tárolásáért, az állapotváltozásokért (mozgás, támadás, interakciók), a játékos bemenetének kezeléséért, valamint a megjelenítésért. Ezek logikailag valóban mind a `Game` osztályhoz kapcsolhatóak, de a funkciók egy része önmagában is értelmes, ezért célszerű leválasztani őket erről az osztályról, és külön-külön egységeket alkotnunk belőlük. Az alábbiakban erre mutatunk egy lehetséges megközelítést.

A megjelenítésért felelős egység

A játékelemek, az ellenfelek és a játékos kirajzolása ugyanazt a gondolatmenetet követi, és mind arra a feltételezésre építenek, hogy a játékot parancssori környezetben valósítjuk meg, amely persze nem az egyetlen lehetőség. Készítsünk egy külön osztályt azokból a funkciókból, amelyek valamilyen parancssorra rajzolást valósítanak meg, így a játékmotort függetleníteni tudjuk a rajzolásra használt "eszköztől", vagyis elvben ezt a komponenst később egy vele funkcionalitásban megegyező, de más technológiát használó komponensre cserélve a játékunk komolyabb átalakítás nélkül is működőképes marad, csak a kirajzolás például valódi grafikus felületen történik.

Hozd létre a `ConsoleRenderer` osztályt az alábbiak szerint.

- Helyezzük át a **RenderSingleSprite** és a **RenderGame** metódusokat ebbe az osztályba. Ez természetesen ideiglenesen hibás kódot eredményez, amit a további lépésekkel korrigálunk.
- Hozzunk létre az osztályban egy `Game` típusú mezőt. Ebben azt a játék-objektumot fogjuk tárolni, amelynek a megjelenítését végzi a `ConsoleRenderer` objektum. A mező a konstruktor hívásakor az átadott `Game` objektumot kapja értékül.
- Alakítsuk át a **RenderGame** metódust úgy, hogy a `GameItem`-ekből álló listát, a `Demon`-okból álló listát, illetve a `Player` példányt az előbbi `Game` objektumon keresztül érje el (vagyis a játék állapotát leíró objektum tulajdonságait lekérve rajzolja ki a játékot), továbbá módosítsuk a láthatóságát publikusra.

Módosítsuk a `Game` osztályt az alábbiak szerint.

- Hozzunk létre egy `ConsoleRenderer` típusú mezőt, amely a játék megjelenítését végző példányt tárolja.
- Az osztály konstruktorában példányosítsuk a `ConsoleRenderer` osztályt, átadva neki a példányosítást kérő `Game` objektumot (használjuk a `this` kulcsszót), majd tároljuk a példányt az előbbi mezőben.
- A **Run** metódusban az előbbi mezőn keresztül hívjuk meg a **RenderGame** metódust.

A logikáért felelős egység

Hozd létre a `GameLogic` osztályt az alábbiak szerint.

- Helyezzük át a **CleanUpGameItems**, a **CleanUpDemons**, **DemonMoveLogic**, a **DemonAttackLogic**, a **GetGameItemsWithinDistance**, a **GetDemonsWithinDistance**, a **GetTotalFillingRatio**, a játékost mozgató **Move**, a démonokat mozgató **Move**, a **PlayerAttackLogic**, illetve az **UpdateDemons** metódusokat ebbe az osztályba. Ez természetesen ismét hibás kódot eredményez, amit a következő néhány egyszerű lépéssel korrigálni tudunk.
- Módosítsuk a játékos és a démonok mozgatását végző **Move** metódusok láthatóságát publikusra.
- Hozzunk létre ebben az osztályban is egy `Game` típusú mezőt, amely a konstruktor hívásakor az átadott `Game` objektumot kapja értékül.
- Az áthelyezett metódusokban hivatkozott `Items` és `Demons` listákat és a `Player` objektumot a `Game` objektumon keresztül érjük el.
- Hozzunk létre egy **UpdateGameState** nevű publikus metódust, amelyben meghívjuk a **UpdateDemons**, **CleanUpGameItems** és **CleanUpDemons** privát metódusokat, ezzel elvégezve a játék állapotmódosítását (amit eddig közvetlenül a `Game` osztály **Run** metódusában kértünk).

Módosítsuk a `Game` osztályt az alábbiak szerint.

- Hozzunk létre egy `GameLogic` típusú mezőt, amely a játék logikáját kezelő objektumot tárolja.

- Az osztály konstruktorában példányosítsuk a `GameLogic` osztályt, átadva neki a példányosítást kérő `Game` objektumot, és tároljuk a példányt az előbbi mezőben.
- A **UserAction** metódusban a **Move** és **PlayerAttackLogic** metódusokat az előbbi mezőn keresztül érjük el.
- A **Run** metódusban az előbbi mezőn keresztül hívjuk meg az **UpdateGameState** metódust.

A frissítési gyakoriság javítása

Jelenleg a játék állapotának frissítése ugyanolyan gyakorisággal történik, mint a játék kirajzolása, azonban ezt érdemes külön időskálákon kezelni.

Módosítsuk a `Game` osztályt az alábbiak szerint.

- Hozzunk létre egy-egy `Stopwatch` típusú példányt (beépített típus, amelyet idő mérésére használhatunk), tároljuk őket egy-egy mezőben.
- A **Run** metódusban indítsuk el a órákat a **Start** metódusuk meghívásával.
- Minden iterációban vizsgáljuk meg, hogy az adott óra `ElapsedMilliseconds` értéke (az indítás óta eltelt idő ezredmásodpercben kifejezve) túllépett-e egy adott értéket. Ha igen, akkor hívjuk a megfelelő metódust (**UpdateGameState** és **RenderGame**), és indítsuk újra az érintett órát a **Restart** metódusának meghívásával (segítségként lásd az alábbi kódrészletet).

```
if (stopwatchLogic.ElapsedMilliseconds > 500)
{
    logic.UpdateGameState();
    stopwatchLogic.Restart();
}
if (stopwatchRenderer.ElapsedMilliseconds > 25)
{
    renderer.RenderGame();
    stopwatchRenderer.Restart();
}
```

Az értékek módosításával beállíthatjuk, milyen gyakorisággal történjen a játéklogika futtatása és a kirajzolás.

Tesztelés

Az előbbi tesztet újra lefuttatva a funkcióknak változatlanul működnie kell, de az ellenfelek mozgása már követhető, valamint a kód áttekinthetőbb és karbantarthatóbb lett.

7 Interakciók kezelése

A játék "szereplői" (tehát a játékos, az ellenfelek és a játékelemek) között bizonyos interakciókat már eddig is létrehoztunk – ilyen volt például a `Door` típusú játékelemek gombnyomásra történő nyitása a játékos által, vagy az ellenfelek és a játékos támadásai –, most ezeket az interakciókat bővítjük és egységesítjük. A támadást már megvalósítottuk előbbiekben, így ebben a részben csak a játékelemekkel történő interakciókkal foglalkozunk.

A játékos interakciói a játékelemekkel

A játékos kétféle módon léphet interakcióba a környező játékelemekkel: szándékos (vagy közvetlen) formában akkor, amikor az interakciót kiváltó billentyűt lenyomja, például mert ki szeretné nyitni a mellette lévő ajtót; illetve közvetett formában akkor, amikor olyan csempére lép, ahol például felvehető vagy sebzést okozó játékelem található.

A közvetlen interakciók kezeléséhez módosítsuk a `GameLogic` osztályt az alábbiak szerint.

- Hozzunk létre egy **PlayerDirectInteractionLogic** nevű publikus metódust. Ebben első lépésként kérdezzük le a játékos pozíciójától 1 távolságnál nem messzebb lévő játékelemeket, majd járjuk be a kapott gyűjteményt. Kezeljük a `Door` és `LevelExit` típusú elemeket úgy, hogy mindkét esetben meghívjuk a játékelem **Interact** metódusát, de `LevelExit` esetén a játék végét jelző `Exited` tulajdonságot állítsuk igazra.

A közvetett interakciók kezeléséhez módosítsuk előbb a `Player` osztályt az alábbiak szerint.

- Hozzunk létre egy **PickUpAmmo** nevű publikus metódust, amely egyetlen egész értéket vár paraméterként. A metódus növelje meg a játékosnál lévő lőszer mennyiségét az átadott értékkel, de ügyeljünk arra, hogy a lőszer mennyisége ne legyen `MaxAmmo`-nál nagyobb.
- Hozzunk létre egy **PickUpHealth** nevű publikus metódust, amely egyetlen egész értéket vár paraméterként. A metódus növelje meg a játékos életerejét az átadott értékkel, de ügyeljünk arra, hogy az életerő ne legyen `MaxHealth`-nél nagyobb.

Módosítsuk a `GameLogic` osztályt az alábbiak szerint.

- Hozzunk létre egy **PlayerIndirectInteractionLogic** nevű publikus metódust. Első lépésként kérdezzük le a játékos pozíciójától 0 távolságra lévő játékelemeket, majd járjuk be a kapott gyűjteményt. Kezeljük az `Ammo`, `Medikit` és `ToxicWaste` típusú elemeket úgy, hogy mindegyik esetben meghívjuk a játékelem **Interact** metódusát, továbbá
  - `Ammo` esetén hívjuk meg a játékos **PickUpAmmo** metódusát (az átadott érték legyen 5),
  - `Medikit` esetén hívjuk meg a játékos **PickUpHealth** metódusát (az átadott érték legyen 25),
  - `ToxicWaste` esetén hívjuk meg a játékos **TakeDamage** metódusát (az átadott érték legyen 5).

Módosítsuk a `Game` osztályt az alábbiak szerint.

- A **userAction** metódusban a játékos interakcióit kiváltó billentyű lenyomásakor hívjuk meg a **PlayerDirectInteractionLogic** metódust.
- A **userAction** metódusban a billentyűlenyomások kezelését követően hívjuk meg a **PlayerIndirectInteractionLogic** metódust.

Az ellenfelek interakciói a játékelemekkel

Az ellenfelek a játékban csak közvetett formában léphetnek interakcióba bizonyos játékelemekkel (tehát például nem képesek kinyitni az ajtókat, vagy lőszert felvenni).

Módosítsuk a `GameLogic` osztályt az alábbiak szerint.

- Hozzunk létre egy **DemonIndirectInteractionLogic** nevű privát metódust, amelynek egyetlen paramétere egy `Demon` példány. Első lépésként kérdezzük le a démon pozíciójától 0 távolságra lévő játékelemeket, majd járjuk be a kapott gyűjteményt. Kezeljük a `ToxicWaste` típusú elemeket úgy, hogy meghívjuk a démon **TakeDamage** metódusát (az átadott érték legyen 5).

- Az **UpdateDemons** metódusban az állapotfrissítés és az állapotok kezelése után hívjuk meg a **DemonIndirectInteractionLogic** metódust az aktuális démon példány átadásával.

Tesztelés

Az előbbi tesztet újra lefuttatva Doomguy fel kell tudja venni a lőszert és az életerő-csomagokat, a mérgező hulladékra lépve pedig mind a játékos, mind az ellenfelek életereje csökkenni fog.

8 Pálya betöltése

A játék pályáit egyszerű szövegfájlként tároljuk, amely tartalmazza az összes játékelem, az ellenfelek és a játékos induló helyzetét is. A formátum könnyen áttekinthető és szerkeszthető, ugyanakkor nem ideális olyan szempontból, hogy a játék aktuális állapotának tárolását (mentését) nem tudjuk megoldani a segítségével. Erre a továbbfejlesztési lehetőségekben teszünk utalást.

A pályát leíró szöveges fájl formátuma

A szöveges fájl egy-egy karakterként kódolja a pálya minden csempéjét, amelyen kezdetben legfeljebb egy játékelem, ellenfél vagy a játékos tartózkodhat. A fájl első sora a pálya méretét tartalmazza vesszővel elválasztva (sorok száma, oszlopok száma). A fájl minden sorának pontosan ugyanannyi karakterből kell állnia, a teljes pálya mérete pedig ne legyen nagyobb a parancssoron egyszerre megjeleníthető sorok és oszlopok számánál.

Az alábbi szöveg egy 7×13 méretű pályát kódol a következő jelölésekkel:

- `A`: `Ammo` típusú játékelem
- `B`: `BFGCell` típusú játékelem
- `D`: `Door` típusú játékelem
- `E`: `LevelExit` típusú játékelem
- `M`: `Medikit` típusú játékelem
- `T`: `ToxicWaste` típusú játékelem
- `W`: `Wall` típusú játékelem
- `z`: `Zombieman` típusú ellenfél
- `i`: `Imp` típusú ellenfél
- `m`: `Mancubus` típusú játékelem
- `p`: játékos
- `_`: üres csempe

```
7,13
WWWWWWWWWWWWW
W_M_WTTTB__pW
W_A_Dz____WWW
W___WTTT__W__
W_m_WWWW__W__
W_E_W__WiiW__
WWWWW__WWWW__
```

A pálya létrehozása a szövegfájl alapján

A pályát a szövegfájl betöltésével és karakterenkénti feldolgozásával fogjuk létrehozni. Módosítsuk a `Game` osztályt az alábbiak szerint.

- Hozzunk létre egy **LoadMapFromPlainText** nevű publikus metódust, amely egy szövegfájl elérési útvonalát várja paraméterként. A fájl első sorából nyerjük ki a pálya dimenziót. Tekintsük a pálya bal felső sarkában lévő csempe koordinátáját $(0, 0)$-nak. Dolgozzuk fel sorról sorra, azon belül karakterről karakterre haladva a szövegfájl sorait (az első kivételével), majd az épp beolvasott karaktertől függően hozzuk létre a megfelelő játékelemet, ellenfelet vagy játékos objektumot, amit az `Items` vagy `Demons` gyűjteményekhez adunk, vagy eltárolunk a `Player`-ben. Az objektum pozícióját az aktuálisan olvasott karakter sor- és oszlopindexének megfelelően állítsuk be.

A főprogramban a `Game` osztály példányosítását követően hívjuk meg a **LoadMapFromPlainText** metódust, átadva a pályát kódoló szöveges fájl elérési útvonalát, majd indítsuk el a játékot a **Run** meghívásával.

Tesztelés

A játékot elindítva a megadott pálya betöltődik, az ellenfelek mozognak és megfelelő közelség esetén támadni kezdenek. A játékelemekkel interakcióba léphetünk (ajtót nyithatunk vagy zárhatunk, a mérgező hulladékra lépve sebződünk, lőszert vagy életpontokat vehetünk fel). A játék véget ér, ha az életerőnk nullára csökken, vagy ha elérjük és átkapcsoljuk a pálya kijáratát jelentő kapcsolót.

9 Hangeffektek

A játékélmény növeléséhez hanghatásokkal bővítjük a játékot. A leírásban szereplő megközelítés *Windows* operációs rendszeren történő fejlesztésre épít, vagyis más operációs rendszerek (pl. *OSX* vagy *Linux*) esetén nem feltétlenül működik az itt megadottak szerint.

A játékban megjelenő hangokat két csoportra oszthatjuk: háttérzenére és hanghatásokra. A játékmotorunk egyidejűleg legfeljebb egy háttérzenét, és ettől függetlenül legfeljebb egy hangeffektet tud lejátszani.

Kapcsoljuk a projekthez a *Windows Media Player* függvénykönyvtárat az alábbiak szerint[1]:

- *Solution explorerben* jobb egérklikk a projekt nevén,
- *Add / Project Reference*,
- válasszuk a *COM* elemet, majd a listában pipáljuk be a *Windows Media Player* elemet,
- végül kattintsunk az *OK* gombra.
- Vegyük használatba a `WMPLib` névteret az `using WMPLib;` utasítással (ezt a `Game` osztályt tartalmazó forrásfájl elején, a névtér deklarációja előtt tegyük meg).

Módosítsuk a `Game` osztályt az alábbiak szerint.

- Hozzunk létre egy-egy `WindowsMediaPlayer` típusú mezőt, az egyiket a háttérzene (`MusicPlayer`, a másikat a hangeffektek (`SfxPlayer`) lejátszására fogjuk használni.
- Az osztály konstruktorában példányosítsuk a `WindowsMediaPlayer` osztályt, tároljunk egy-egy példányt a két mezőben.
- Definiáljuk a lejátszandó hangeffektek típusát egy `SoundEffectType` felsorolással.

[1] A lépések a Visual Studio fejlesztőkörnyezetet veszik alapul.

```
enum SoundEffectType { BFG, Door, ItemPickup, Pain, PlayerDeath, Shotgun }
```

- Hozzunk létre egy **PlaySoundEffect** nevű publikus metódust, amely paraméterként egy `SoundEffectType` értéket vár. A kapott értéktől függően állítsuk `SfxPlayer` objektum `URL` tulajdonságát a megfelelő hangeffektet tartalmazó fájl elérési útvonalra, majd a `controls.play` metódus meghívásával játszuk le a hangot (lásd az alábbi kódrészletet).

```
switch (effectType)
{
    case SoundEffectType.Door:
        SfxPlayer.URL = "sound/door.mp3";
        break;
    case SoundEffectType.ItemPickup:
        SfxPlayer.URL = "sound/item_pickup.mp3";
        break;
}
SfxPlayer.controls.play();
```

- Hozzunk létre egy **PlayMusic** nevű publikus metódust, amely paraméterként egy zenefájl elérési útvonalát várja. Az előző metódushoz hasonló módon állítsuk be a `MusicPlayer` objektum `URL` tulajdonságát az átadott értékre, majd indítsuk el a lejátszást.

Módosítsuk a `GameLogic` osztályt az alábbiak szerint.

- A **PlayerDirectInteractionLogic**, **PlayerIndirectInteractionLogic** metódusokban hívjuk meg a **PlaySoundEffect** metódust a megfelelő hanghatás típus átadásával.
- A **PlayerAttackLogic** metódusban lövés esetén hívjuk a **PlaySoundEffect** metódust, paraméterként `SoundEffectTyp` értéket átadva.
- A **DemonAttackLogic** metódusban sebzés esetén hívjuk a **PlaySoundEffect** metódust, paraméterként `SoundEffectType.Pain` értéket átadva.

Módosítsuk a `Game` osztályt az alábbiak szerint.

- A **Run** metódusban (a fő ciklus indítása előtt) hívjuk meg a **PlayMusic** metódust, paraméterként a lejátszandó háttérzenét átadva.
- A ciklusból kilépve vizsgáljuk meg, mi miatt ért véget a játék. Ha a játékos teljesítette a pályát, a **PlayMusic** metódus meghívásával váltsunk háttérzenét. Ha a játékos meghalt, hívjuk a **PlaySoundEffect** metódust, paraméterként `SoundEffectType.PlayerDeath` értéket átadva.
- A **Run** metódus vége előtt helyezzünk el egy `Console.ReadLine` utasítást (ellenkező esetben a megfelelő hangeffekt lejátszása után azonnal véget ér a program).

Tesztelés

A játék indításakor megszólal a háttérzene, támadáskor, sebződéskor és interakció esetén a megfelelő hangeffektet halljuk.

10 Befejező lépések

A BFG használata

A *BFG9000* egy zöld plazmát kibocsátó fegyver, amely jelentős sérülést okoz a játékos környezetében lévő ellenfeleknek. Az elsütéséhez energiacellára (`BFGCell`) van szükségünk.

Módosítsuk a `Player` osztályt az alábbiak szerint.

- Az `Ammo`-hoz hasonlóan tároljuk a játékosnál lévő energiacellák értékét, amely legyen lekérdezhető egy `BFGCells` tulajdonságon keresztül.
- Vezessük be a `MaxBFGCells` tulajdonságot is, amely mindig 3-at ad vissza.
- Hozzunk létre egy **PickUpBFGCell** metódust, amely a lőszer felvételéhez hasonló elven működik, és a `BFGCells` értékét lehet megnövelni vele.
- Hozzunk létre egy **ShootBFG** metódust, a hagyományos lövéshez hasonlóan.

Módosítsuk a `GameLogic` osztályt az alábbiak szerint.

- Hozzunk létre egy **PlayerBFGAttackLogic** metódust a játékos támadását megvalósító metódushoz hasonlóan. A lövés hatására minden, a játékos látótávolságán belül lévő ellenfél 100 és 800 közötti véletlen sebzést szenved el. Az elpusztított ellenfelek után azonosan 1 tapasztalati pontot kapjon a játékos.
- A **PlayerIndirectInteractionLogic** metódust bővítsd úgy, hogy az kezelje a `BFGCell` típusú játékelemeket is. Hasonlóan a lőszer felvételéhez, a játékelemre lépve a játékos felveszi az energiacellát (a nála lévő energiacellák száma 1-gyel nő), ha a maximálisnál kevesebb cella van nála.

Módosítsuk a `Game` osztályt az alábbiak szerint.

- A **UserAction** metódust bővíts úgy, hogy az kezelje az `S` billentyű lenyomását is. Ennek hatására hívd meg a **PlayerBFGAttackLogic** metódust.

Állapotjelző csík (heads-up display)

Az állapotjelző csík (HUD) játékos életerejét, a lőszer és energiacella mennyiségét, illetve a tapasztalati pontjait mutatja.

Módosítsuk a `ConsoleRenderer` osztályt az alábbiak szerint.

- Hozzunk létre egy **RenderHUD** nevű privát metódust. Készítsünk egy formázott karakterláncot a játékos adatainak lekérdezésével, amelyet írjunk ki a parancssor alsó sorába (a jobb láthatóság érdekében egységes háttérszínnel).
- A **RenderGame** metódusban a játékelemek, ellenfelek és játékos kirajzolását követően hívjuk meg a **RenderHUD** metódust.

A frissítési idő figyelembe vétele az ellenfelek mozgásánál

A `Game` osztály **Run** metódusában lévő ciklusban a játéklogika frissítését és a kirajzolást adott időközönként végezzük csak el, azonban a játéklogikában eddig nem vettük figyelembe a két frissítési lépés között ténylegesen eltelt időt. Emiatt az ellenfelek mozgása nem tükrözi a 2. táblázatban megadott gyorsaságukat, továbbá a játékos valóságban tapasztalható sebződésének nagysága is attól függ, milyen gyakorisággal hívjuk a játéklogika frissítését (vagyis például rövidebb idő megadásakor az egész játékmenet felgyorsul).

Módosítsuk a `Demon` osztályt az alábbiak szerint.

- Tároljuk a démon gyorsaságát egy egész típusú mezőben, amelynek értéke legyen lekérdezhető egy `Speed` nevű tulajdonságon keresztül.
- A **SetInitialProperties** metódusban állítsuk be a démon típusának megfelelően a gyorsaságát is.

A démonok gyorsaság-értékét értelmezzük úgy, hogy a két szomszédos csempe közötti távolságnak ennyi százalékát teszik meg átlagosan egy másodperc alatt. Például ha egy démon gyorsasága 100, akkor átlagosan minden másodpercben átlép egy szomszédos csempére (ha nincs a közelében akadály), míg egy 25 sebességű démonnak átlagosan négy másodperc kell, hogy átlépjen egy szomszédos csempére. Ezt a gondolatmenetet fogjuk megvalósítani a mozgatást végző logikában.

Módosítsuk a `GameLogic` osztályt az alábbiak szerint.

- Vezessünk be egy hosszú egész (`long`) típusú paramétert a démonok mozgatását végző **Move** metódusnál. Az átadott érték a két mozgatási lépés között eltelt időt jelenti ezredmásodpercben. A démon mozgásának gyorsaságát úgy fogjuk szabályozni, hogy nem feltétlenül mozdítjuk el minden metódushíváskor, hanem ezt a gyorsaságából számított valószínűséggel tesszük csak a fenti ötlet szerint. Ehhez
  - először számítsuk ki az elmozdulás valószínűségét a
    $$P_{\text{Move}} = \frac{\text{Speed}}{100} \cdot \frac{\Delta t}{1000}$$
    képlettel (az adott démon gyorsaságát és a paraméterként átadott Δt időtartamot felhasználva, és ügyelve arra, hogy ne egészek közötti osztás valósuljon meg),
  - majd generáljunk egy 0 és 1 közötti véletlen lebegőpontos értéket a `Random`.**NextDouble** metódus hívásával,
  - és csak akkor futtassuk le a mozgatást végző utasításokat, ha a kapott véletlen érték kisebb P_{Move}-nál.
- Vezessünk be egy hosszú egész (`long`) típusú paramétert a **DemonMoveLogic**, az **UpdateDemons** és az **UpdateGameState** metódusoknál, és módosítsuk a metódushívási-láncot ennek megfelelően.

Módosítsuk a `Game` osztályt az alábbiak szerint.

- A **Run** metódusban az **UpdateGameState** methívásakor adjuk át a két frissítési lépés között eltelt időt.

A frissítési idő figyelembe vétele az ellenfelek támadásánál

Az ellenfelek mozgásához hasonlóan a támadásnál okozott sebzés nagyságát is célszerű összekapcsolnunk a frissítések közötti időtartammal. A 2. táblázatban megadott sebzést értelmezzük úgy, hogy az adott tartományban lévő sebzést egy másodperces időtartamra vetítjük.

Módosítsuk a `GameLogic` osztályt az alábbiak szerint.

- Vezessünk be egy hosszú egész (`long`) típusú paramétert a démonok támadását megvalósító végző **DemonAttackLogic** metódusnál. A véletlenszerűen generált sebzés értéket szorozzuk be a
  $$\frac{\Delta t}{1000}$$
  hányadossal (a frissítési lépések között eltelt időt itt is Δt jelöli).

Csak a látótávolságon belüli elemek megjelenítése

Nagyobb méretű pálya (sok játékelem vagy démon) esetén a kirajzolás lelassul, a megjelenítés szakadozottá válik. Részben emiatt, részben a valószerűbb játékélmény miatt (a játékos egyidejűleg csak a pálya egy kisebb részét

láthatja) alakítsuk át úgy a megjelenítést, hogy csak a játékos látótávolságán belüli elemeket vegyük figyelembe a kirajzolásnál.

Módosítsuk a `ConsoleRenderer` osztályt az alábbiak szerint.

- A **RenderGame** metódusban az `Items` és a `Demons` bejárásakor számítsuk ki az aktuálisan elért elem vagy démon, illetve a játékos távolságát. Csak akkor hívjuk a **RenderSingleSprite** metódust, ha az adott elem nincs kívül a játékos látótávján (használjuk a `SightRange` tulajdonságot).

Tesztelés

Elkészültünk. A játék indításakor megjelenik a betöltött pálya, megszólal a háttérzene, és minden megvalósított funkció használható.

11 Továbbfejlesztési lehetőségek

A játék kódjának megtervezése során igyekeztünk azt úgy kialakítani, hogy valamelyest kövesse a szoftverfejlesztésben általánosan használt *bevált módszereket*. Ennek ellenére természetesen számos lehetőségünk van a funkciók és a szoftver felépítésének javítására.

Ismert bugok

A játék jelenlegi verziója számos olyan jellemzővel rendelkezik, amit *bug*-ként azonosíthatunk. Ilyen például, hogy miközben a játékos csak a négy szomszédos csempe (északi-déli-keleti-nyugati) valamelyikére léphet egy mozgatási lépés során, a démonok mozgásánál javasolt naiv algoritmus az átlós lépéseket is lehetővé teszi, ami miatt bizonyos esetekben a démonok látszólag "átmennek" a falon (amikor ez egy átlós lépéssel megvalósulhat).

Egy másik ismert bug, hogy miközben a démonok sebzése frissítési gyakoriságtól függően változik, a játékos olyan gyakorisággal adhat le ugyanolyan sebzést okozó lövéseket, amilyen gyakorisággal a fő ciklusban a felhasználói bemenetet vizsgáljuk (ez jelenleg közelítőleg 40 FPS, vagyis elvben másodpercenként akár 40 lövést is leadhatunk), ami aránytalan előnyt jelent a játékosnak.

Szintén a támadáshoz köthető bug, hogy mind a játékos, mind a démonok képesek falon keresztül is sebzést okozni. Jellegében hasonló furcsaság még, hogy a játékos látótávolságán belül minden elemet és démont kirajzolunk, akkor is, ha az a játékos szemszögéből egy fal vagy zárt ajtó mögött található.

Néhány ötlet a továbbfejlesztésre

Könnyen látható, hogy a `Player` és `Demon` osztályokban sok a közös adattag, amiből következően például a mozgatásukat végző **Move** metódusok is nagyrészt hasonló logikára épülnek. Épp ezért, *öröklés* használatával megoldható volna a két osztály közös ősből való származtatása, ami egyszerűsítené a felépítésüket és a rajtuk végzett működési logikát is.

A démonok típusát jelenleg a `DemonType` felsorolással definiáljuk, öröklés használatával viszont a `Demon` osztályból származtathatnánk a konkrét démon-fajtákat. Ezzel lehetőségünk volna például az adott típusú ellenfelet reprezentáló osztályban definiálni az egyedi jellemzőket (mint a gyorsaság, vagy a sebzési értékek tartománya), így a `GameLogic` osztályban egységesebben dolgozhatnánk velük (ott nem kellene elágazásokkal kezelnünk a típustól függő sebzést vagy az utánuk járó tapasztalati pontokat).

Hasonló elv jelenik meg a `ConsoleRenderer` osztály **RenderGame** metódusában is, amely jelenleg három különböző, de bizonyos tulajdonságok szerint mégis hasonló adattal dolgozik: a játékelemekkel, a démonokkal és a játékossal. Ezek a megjelenítés szempontjából egyformák viszont abban az értelemben, hogy mind *megjeleníthetőek*. Ez lehetőséget biztosítana arra, hogy együtt, valamilyen absztrakció mögé rejtve kezeljük őket, például egy `IVisualizable` interfész bevezetésével, amely egy osztálynál a megjeleníthetőségért felelős képességeket írja elő. Ezt bevezetve a **RenderGame** metódusnak egyetlen `IVisualizable` interfészt megvalósító példányok gyűjteményét kellene csak átadnunk a helyes működéshez.

Egy egészen egyszerű módosítási lehetőség még a kódban használt, de jelenleg más-más helyen létrehozott felsorolásokat (enumokat) egyetlen fájban tárolni, így egységesítve a kód szervezését.

Zárásképp pedig egy általános igazság: egy szoftver soha nincsen kész, csak elég stabillá válik ahhoz, hogy ne kelljen azonnal újraírni.

"Rip and tear, until it is done."